

Plano — Correção dos KPIs de Monitoring: período, unidades e escala

Data: 2026-08-21 **Origem:** análise de conformidade das fórmulas 4G/5G contra `Fórmulas_4G_Monitoring(1).txt` , `Fórmulas_5G_Monitoring.txt` , os CSVs de referência em `csvs_reference/` e os HARs em `har-5g-oss/` . **Respostas do produto:** `answers_KPIS.md` .

Contexto

As fórmulas implementadas em `core/kpi_formulas.py` são **algebricamente fiéis** aos dois documentos (11/11 no 4G, 13/13 no 5G). A validação cruzada — recalcular os KPIs a partir do CSV do OSS e comparar com o banco, mesma célula e mesmo minuto — deu razão **1,000** em todas as métricas, com uma exceção: `availability` no 5G, razão 0,2.

Os problemas encontrados não estão na álgebra. Estão em **três decisões de contorno** tomadas quando um dado do OSS não estava claro, e em **uma camada de apresentação** que nunca recebeu tratamento de unidade.

Achados que este plano endereça

id	achado	evidência
A1	<code>results[].period = 5</code> foi adotado como Granularity Period; o GP real é 1 min	CSV <code>Period(minute)=1</code> , <code>N.Cell.Avail.Dur=60 s</code> , <code>execTime</code> de 60 em 60 s
A2	<code>N.ThpTime.*</code> chega sem unidade; throughput 5G sai ~1000× menor	79/83 amostras DL e 82/90 UL violam o piso <code>volume/60 s</code> ; valores múltiplos de 500 (slot de 0,5 ms)
A3	4 células 4G com <code>PRB.Avail</code> de 150/225, fora do padrão LTE	<code>4G-SPSMH1-18-DA/DB/DI</code> , <code>21-DB</code>
A4	RTT 4G nunca calcula (contadores fora da task, limite de 25 no OSS)	0 linhas no banco; 56 <code>invalid_formula</code> por ciclo
A5	<code>traffic_volume_dl_sa</code> negativo	1 amostra em 2.135; presente também no CSV do OSS
B1	volume gravado em 3 bases diferentes sob o mesmo <code>metric</code>	4G em <code>bit</code> , 5G em <code>kbit</code> , banco legado em <code>MB</code>
B3	painéis pareados usam dois eixos mesmo com unidade idêntica	Traffic Volume NSA: barra do mesmo tamanho vale 8× menos
B4	sem escala automática; amplitude de 7–8,5 ordens de grandeza	<code>traffic_volume_dl</code> : 320 bit ... 5,84 Gbit
B6	acessibilidade/drop 5G só calculam em ~19% dos minutos	80,0% e 82,1% das amostras sem denominador, no CSV do OSS
B7	acessibilidade com denominador mínimo dispara CRITICAL	454 alertas críticos; valores de 50,0% / 66,7% / 80,0% (1 de 2, 2 de 3, 4 de 5)
B9	throughput de site é soma de taxas	site 4G com 658 Mbit/s

O que a coleta não tem de errado

Auditoria dos três CSVs: grade de 1 minuto **100% completa**, zero buracos, zero duplicatas, inventário casando 28/28 células 4G e 5/5 células 5G. **Todo buraco nos gráficos é gerado pelas fórmulas do app, nenhum é perda de coleta.** Isso delimita o escopo: não há trabalho de confiabilidade de coleta neste plano.

Divisão em fases

Quatro fases. A divisão segue as dependências reais, não o tamanho:

```
Fase 1  Fórmulas e coleta corretas (backend puro)
      |
      +--> Fase 2  Período da task e unidade do threshold (UI do servidor)
      |
      +--> Fase 3  Unidade canônica na leitura
                  |
                  +--> Fase 4  Apresentação: escala automática e eixo único
```

- **Fase 1 é pré-requisito de tudo** — não faz sentido formatar bonito um número errado.
- **Fase 3 depende da Fase 1** porque o fator canônico do throughput 5G só é conhecido depois que a Fase 1 resolve a unidade de tempo.
- **Fase 4 depende da Fase 3** porque a escala automática lê a unidade-base declarada.
- **Fase 2 é independente da 3 e da 4** e pode ser feita em paralelo. A Fase 1 já funciona sem ela, assumindo o default de 1 minuto (que é o valor real de todas as tasks hoje, conforme A1).

Ponto que continua sem confirmação: a unidade de `N.ThpTime.*` (A2)

A resposta do A2 confirmou o que mais importava — **DL e UL usam a mesma unidade**, o que tira `throughput_ul` do limbo de `production_ready=False`. Mas a unidade absoluta (“segundo ou milissegundo”) não fecha aritmeticamente:

```
N.ThpTime.UE.UL.RmvSmallPkt = 36.079.000, em um período de 60 s
se segundo      -> 417 dias de tempo escalonado  IMPOSSÍVEL
se milissegundo -> 10 horas                      IMPOSSÍVEL
se microssegundo -> 36,1 s                       possível
```

Somado a: 95/95 valores são múltiplos de 500 (o slot de 0,5 ms em 30 kHz SCS) e o piso `throughput ≥ volume/60 s` é violado em 79/83 amostras DL sob qualquer outra hipótese.

Decisão do plano: implementar microssegundo como constante nomeada e documentada, e **instalar uma trava aritmética que verifica o piso a cada cálculo**. Se a unidade for outra, a trava acende no painel de coleta em vez de o sistema publicar número errado em silêncio. A constante é o único ponto a mudar caso a documentação Huawei diga outra coisa.

Fase 1 — Fórmulas e coleta corretas

O que será desenvolvido

Esta fase conserta os números na origem, sem tocar em nenhuma tela. Ao final, o valor que o app grava passa a bater com o OSS em **todas** as métricas — hoje bate em todas menos `availability` do 5G. Também para de marcar todo ciclo como “Parcial” por um motivo que não é falha.

São sete correções independentes no `core/`, todas de baixo risco e cobertas por teste unitário.

Escopo detalhado

Código

`core/kpi_formulas.py`

1. Nova exceção `NotApplicable(InvalidKpi)`. Subclasse, para que todo `except InvalidKpi` existente continue funcionando. Passa a ser levantada por `_need` (contador ausente na task) e por `_ratio` (denominador zero). Fica reservado ao `InvalidKpi` puro o que é defeito de verdade: resultado não numérico.

2. A2 — fator de tempo do throughput 5G.

```
# Medido, não documentado: o OSS devolve N.ThpTime.* com unit vazio. Todos os
# valores são múltiplos de 500 (slot de 0,5 ms @30 kHz) e a hipótese de ms/s
# viola o piso volume/periodo em 79 de 83 amostras. Ver plano 2026-08-21-001.
THP_TIME_TO_SECONDS = 1e-6
```

`_n_thp_dl` e `_n_thp_ul` passam a devolver **Mbit/s** de verdade ($\text{kbit}/\mu\text{s} \rightarrow \text{Gbit/s} \rightarrow \times 1000$). `throughput_ul` do `5G_NRDUCELL` recebe `unit="Mbit/s"` e `production_ready=True`.

3. A2 — trava aritmética. Nova função `check_throughput_floor(definition, counters, period, value)`: para as métricas de throughput, o resultado tem de ser $\geq \text{volume}/(\text{period} \times 60)$. Violação **não descarta a linha** — devolve um motivo que o parser transforma em diagnóstico `unit_suspect`. Escolha deliberada: durante evento ao vivo, dado suspeito sinalizado é melhor que dado ausente.

4. A5 — clamp do volume SA. `_n_volume_dl_sa` e `_n_volume_ul_sa` passam a devolver $\max(0.0, \text{total} - \text{nsa})$, com log em `DEBUG` quando clampa. Confirmado como bug do OSS: a inconsistência está no próprio export CSV.

5. A4 — RTT indisponível. `ran_rtt` e `terrestrial_rtt` recebem `monitoring_available=False`. Comentário registrando o motivo (limite de 25 contadores por task no OSS) para não serem “reativadas por engano” depois.

6. B9 — throughput de site. `site_aggregation` de `throughput_dl` / `throughput_ul`, nas duas tecnologias, passa de `"sum"` para `"recalculate"`. O parser já sabe recalculá-lo a partir dos contadores somados ($\Sigma(\text{vol-last})/\Sigma\text{tempo}$), então é mudança de uma palavra por linha. **Descontinuidade conhecida:** linhas SITE antigas são somas e as novas são recálculos. Como o B1 decidiu não migrar histórico, isso fica documentado no `MEMORY.md`, não corrigido.

7. B7 — tamanho de amostra. Nova função `sample_size(definition, counters) -> float | None`, que devolve o **menor denominador** da fórmula para `accessibility` e `drop_rate`, e `None` para as demais. Não altera nenhum cálculo; só expõe o dado.

core/collector.py

8. A1 — o período vem da configuração, não da resposta. `_configured_pm_tasks` passa a ler `period_seconds` de cada item de `integration.pm_tasks`, com **default 60**. `_parse_monitoring_response` usa esse valor no lugar de `result.get("period")`. O `period` da resposta continua sendo lido, mas apenas para um **aviso de divergência** no log — é um dado do OSS e some-lo seria perder rastro.

9. A3 — aviso de `PRB.Avail` fora do padrão. Ao montar os contadores, se `L.ChMeas.PR.B.DL.Avail` / `.UL.Avail` não estiver em `{6, 15, 25, 50, 75, 100}`, logar `WARNING` uma vez por célula por sessão, nomeando a célula e o valor. **Nenhuma fórmula muda** — a resposta do A3 foi “faça como achar melhor”, e as duas hipóteses (SFN somado vs. denominador inflado) não são decidíveis com os dados coletados. O aviso existe para que a próxima ocorrência apareça em vez de passar 9 dias despercebida.

10. B6/A4 — separar “não aplicável” de “inválido”. `_parse_monitoring_response` passa a contar `not_applicable` separado de `invalid`. O cálculo de `partial` deixa de incluir `not_applicable`. Consequência direta: um ciclo 4G saudável passa a fechar “Com dados” em vez de “Parcial” permanente.

11. B7 — carregar o tamanho da amostra. Cada medição em memória ganha `m["sample_size"]` (não persistido, não vai para o banco). É consumido pelo agendador no mesmo ciclo.

core/collection_result.py

12. Novo campo `not_applicable: int = 0`, com a mesma validação de não-negativo dos demais.

core/scheduler.py

13. **B7 — piso de amostra no alarme.** `_evaluate_kpi_alerts` passa a ignorar o alerta de `accessibility` quando `m.get("sample_size")` for menor que `thresholds.get("alert_min_samples", 20)`.

Ampliação em relação ao B7 original, com justificativa. A pergunta B7 foi feita sobre `drop_rate`. Ao implementar, verifiquei que `drop_rate` não gera alerta nenhum hoje — `_evaluate_kpi_alerts` só trata `utilization_dl` e `accessibility`. O falso positivo real está na acessibilidade: valores de 50,0% (6 ocorrências), 66,7% e 80,0% são a assinatura de denominadores 2, 3 e 5, e cada um deles virou um `CRITICAL`. O piso é aplicado onde o problema existe. Quando `drop_rate` ganhar alerta, herda o mesmo piso.

Efeito colateral aceito e explícito: com piso 20 e granularidade de 1 minuto, o 5G deixa de gerar alerta de acessibilidade (o maior denominador observado por célula-minuto é 4). É consistente com o volume de tráfego SA medido — 0,049% do total. A Fase 4 do B6 (agregação por janela) multiplica o denominador por ~15 e devolve o alarme ao 5G.

`tests/test_kpi_formulas.py`

14. **C2 — corrigir o teste que fixa o comportamento errado.** `test_availability_requires_the_period_returned_by_oss` é renomeado para `test_availability_requires_the_configured_granularity_period` e passa a usar `period=1` nos vetores, refletindo o GP real.

Interface

Nenhuma tela nova. Duas mudanças visíveis caem de graça:

- O **painel de coleta** para de exibir “Parcial” permanente nos ciclos 4G saudáveis.
- O contador de `inválidos` no rodapé do painel (`frontend/js/app.js:1018`) passa a mostrar só defeito real. O contador de “não aplicável” é exibido na Fase 4, junto com a rotulagem do B6.

Testes

arquivo	teste	o que trava
test_kpi_formulas.py	test_availability_uses_configured_period	N.Cell.Avail.Dur=60 com period=1 → 100%, e com period=5 → 20% (o bug de hoje)
test_kpi_formulas.py	test_4g_availability_reports_zero_for_fully_down_cell	Unavail.Dur=60 , period=1 → 0%. Com o bug daria 80%
test_kpi_formulas.py	test_5g_throughput_is_in_mbit_per_second	vetor real do CSV → 318,4 Mbit/s , não 0,318
test_kpi_formulas.py	test_throughput_floor_flags_suspect_time_unit	valor abaixo de volume/período devolve motivo unit_suspect
test_kpi_formulas.py	test_traffic_volume_sa_never_negative	N.NSA.ThpVol.DL > N.ThpVol.DL → 0.0 , nunca negativo
test_kpi_formulas.py	test_rtt_metrics_are_not_offered_by_monitoring	ran_rtt / terrestrial_rtt fora de catalog_for_api()
test_kpi_formulas.py	test_site_throughput_recalculates_from_summed_counters	duas células → $\Sigma(vol-last)/\Sigma tempo$, não a soma das taxas
test_kpi_formulas.py	test_sample_size_returns_smallest_denominator	acessibilidade com Att=2 → 2 ; utilization_dl → None
test_kpi_monitoring.py	test_period_comes_from_task_config_not_response	resposta com period=5 + config period_seconds=60 → availability 100%
test_kpi_monitoring.py	test_missing_counter_is_not_applicable_not_invalid	contador ausente → not_applicable=1 , invalid=0 , ciclo não parcial
test_kpi_monitoring.py	test_zero_denominator_is_not_applicable	idem para denominador zero
test_kpi_monitoring.py	test_non_standard_prb_avail_logs_warning	PRB.Avail=225 → um WARNING com o nome da célula (via caplog)
test_scheduler.py	test_accessibility_alert_requires_minimum_sample	value=50.0, sample_size=2 → nenhum alerta; sample_size=40 → alerta

Como validar e testar

Testes automatizados

```
python -m pytest tests/ -q --basetemp=.pytest-work/fase1
```

Critério: 0 falhas, e a suíte não pode encolher — o baseline registrado no MEMORY.md é 388 passed / 10 skipped.

Validação contra o dado real (obrigatória nesta fase)

O ativo mais valioso desta análise é poder conferir contra o OSS sem VPN. Criar tools/kpi_crosscheck.py , que recalcula os KPIs a partir dos CSVs de csvs_reference/ usando core.kpi_formulas e compara com o banco do evento, célula a célula e minuto a minuto:

```
python tools/kpi_crosscheck.py --event testesantoamaro --csv-dir csvs_reference
```

Saída esperada depois da Fase 1 (hoje availability sai com razão 0,2):

```
5G_NRCCELL 5G-SPSMG7-35-MB 2026-08-21T11:10:00Z
availability OSS=100.000 APP=100.000 razão=1.000 OK
user_count OSS= 19.356 APP= 19.356 razão=1.000 OK
4G 4G-SPSMH2-18-DC 2026-08-21T11:10:00Z
accessibility OSS=100.000 APP=100.000 razão=1.000 OK
throughput_dl OSS= 6.776 APP= 6.776 razão=1.000 OK
...
RESULTADO: 22/22 métricas com razão 1,000 (tolerância 1e-6)
```

Este script fica no repositório — é a rede de segurança para as Fases 3 e 4, que mexem em unidade e não podem quebrar o valor.

Validação visual

Abrir o app com o evento `testesantoamaro`, aba **Visão geral de KPIs** → 5G:

painel	antes	depois
Availability	linha reta em 20%	linha reta em 100%, acima do threshold
Throughput DL	pico ~1,0	pico ~1.000 (número feio; a Fase 4 formata)
Traffic Volume SA	mergulha abaixo de zero	nunca cruza o zero

E no painel de coleta: um ciclo 4G bem-sucedido deve exibir “Com dados”, não “Parcial”.

No log (`data/logs/smart_events.log`), conferir a nova linha do A3:

```
[monitoring] PRB.Avail fora do padrão LTE em 4G-SPSMH1-18-DA: DL=225 UL=225
(esperado 6/15/25/50/75/100) – utilização pode estar subestimada
```

Sugestão de commit

```
fix: correct monitoring KPI period, 5G throughput unit and alert sampling
```

Fase 2 — Período da task e unidade do threshold no cadastro

O que será desenvolvido

A Fase 1 passou a ler o Granularity Period da configuração do evento, com default de 1 minuto. Esta fase cria o campo para o operador informar esse período ao vincular a task, atendendo ao pedido do A1: **um input inteiro + um dropdown de s / min / h**. Hoje todas as tasks são de 1 minuto, mas existem tasks de 15 min e 1 h no OSS, e sem o campo elas produziriam availability errada em silêncio — exatamente a falha que a Fase 1 acabou de fechar.

Junto vem o B2: gravar a **unidade ao lado de cada threshold**, para que uma futura mudança de base não invalide alarme nenhum sem avisar.

Escopo detalhado

Código

`server_frontend/index.html` (formulário de cadastro de evento)

- 1. `addPmTaskRow(tech, taskId, periodValue, periodUnit)` ganha dois controles na mesma linha: um `input[type=number]` (`.pm-task-period-value`, default `1`, `min=1`) e um `select` (`.pm-task-period-unit`) com `s / min / h`, default `min`.
- 2. `configuredPmTasks(integration)` passa a normalizar para `period_seconds` — uma única unidade no JSON, evitando que o backend tenha de reinterpretar. A UI converte na leitura e na escrita; o contrato guarda segundos.
- 3. Item sem `period_seconds` (todo evento já cadastrado) é lido como `60`. Sem migração de banco: a normalização acontece na leitura do config.
- 4. **B2 — threshold com unidade.** O bloco de thresholds passa a gravar `{"value": 80, "unit": "%"}`.
`Api._metric_thresholds` aceita as duas formas — número cru (legado) e objeto — e devolve sempre o objeto normalizado.

core/collector.py

- 5. `_configured_pm_tasks` valida `period_seconds`: inteiro positivo; ausente ou inválido cai para 60 com `WARNING` nomeando a task.

api/api.py

- 6. `_metric_thresholds` passa a devolver `{"value", "unit"}` em vez de número cru. Os consumidores (`get_kpi_series`, `get_kpi_overview`, `_evaluate_kpi_alerts`) leem `.value`.

Interface

Linha de task PM no cadastro, antes e depois:

antes [4G / LTE v] [Task ID _____] [x]
depois [4G / LTE v] [Task ID _____] [1] [min v] [x]

O rótulo da lista de eventos passa a informar o período quando ele não for o default: `4G x3 · NR Cell x1 (15 min)`.

Testes

arquivo	teste	o que trava
test_server_frontend_kpi_config.py	test_pm_task_row_has_period_value_and_unit	presença de <code>.pm-task-period-value</code> e <code>.pm-task-period-unit</code> no HTML
test_server_frontend_kpi_config.py	test_pm_task_period_is_persisted_in_seconds	<code>period_seconds</code> no payload de gravação
test_server_frontend_kpi_config.py	test_legacy_task_without_period_defaults_to_60s	task sem o campo é renderizada como <code>1 min</code>
test_kpi_monitoring.py	test_task_period_of_15_minutes_changes_availability	<code>period_seconds=900</code> + <code>Avail.Dur=900</code> → <code>100%</code>
test_kpi_monitoring.py	test_invalid_period_seconds_falls_back_to_60_with_warning	<code>period_seconds="abc"</code> → <code>60</code> + <code>WARNING</code>
test_api.py	test_metric_thresholds_accepts_legacy_number_and_object	<code>80</code> e <code>{"value":80,"unit":"%"}</code> devolvem o mesmo objeto

Como validar e testar

Testes automatizados

```
python -m pytest tests/test_server_frontend_kpi_config.py tests/test_kpi_monitoring.py tests/test_api.py -q
python -m pytest tests/ -q --basetemp=.pytest-work/fase2
```

Validação visual

1. Subir o servidor de cadastro e abrir o formulário de um evento novo. A linha de task deve nascer com `1 + min`.
2. **Teste de regressão de cadastro:** abrir `TesteSantoAmaro` para edição. As quatro tasks (747, 748, 749, 753) devem aparecer com `1 min` preenchido, sem o operador ter feito nada. Salvar sem alterar e reabrir — os valores devem persistir.
3. **Teste do caminho novo:** trocar uma task para `15 min`, salvar, e conferir no `data/smart_events.db` que o `config_json` gravou `"period_seconds": 900`.
4. Reverter para `1 min` antes de seguir.

Sugestão de commit

```
feat: configure monitoring task granularity and threshold units per event
```

Fase 3 — Unidade canônica na leitura

O que será desenvolvido

Hoje a mesma métrica está gravada em bases diferentes conforme a origem: `traffic_volume_d1` do 4G está em `bit`, os `traffic_volume_*` do 5G em `kbit`, e o banco legado `vips-rio-tim-jun-2026.db` (1.011.298 linhas, sem coluna `technology`) em `MB`. Nada na linha denuncia isso — a unidade é implícita no par `(metric, technology)` e vive só no catálogo.

Conforme o B1, o **histórico não será migrado**. A unidade passa a ser declarada no catálogo e convertida na leitura, num único ponto. O armazenado continua idêntico ao que o OSS entrega, o que preserva a auditoria direta contra os CSVs que o `kpi_crosscheck.py` faz.

Escopo detalhado

Código

```
core/kpi_formulas.py
```

1. `KpiDefinition` ganha dois campos:

```
base_unit: str = ""          # unidade canônica exposta ao frontend: "bit", "bit/s", "%", ...
to_base: float = 1.0        # fator do valor gravado para a unidade canônica
```

Preenchimento: volume 4G (`"bit", 1.0`) ; volume 5G (`"bit", 1e3`) ; throughput 4G e 5G (`"bit/s", 1e6`) . Percentuais, dBm , ms e usuários ficam com `to_base=1.0` .

2. Nova função pública:

```
def to_canonical(metric: str, technology: str | None, value: float) -> float
```

Sem definição correspondente (linha legada sem `technology`), devolve o valor **intacto** e não inventa conversão. Documentado no docstring: bancos anteriores à coluna `technology` não são normalizáveis e ficam fora do contrato.

3. **B8 — o contador legado.** `traffic_volume_dl` / `traffic_volume_ul` do 4G:

- nome passa de "Volume de Tráfego DL (legado)" para "Volume de Tráfego DL" — "legado" descreve a origem do código, não o dado, e não diz nada ao operador;
- `unit` passa de "contador OSS" para "bit" ;
- `monitoring_available` continua **True** . Removê-las do seletor mataria a coluna "Participação" da lista de sites, que só aparece quando a métrica está selecionada (`frontend/js/kpi.js:349-351`).

core/database.py

4. As funções de leitura — `get_kpi_series` , `get_kpi_series_by_cell` , `get_kpi_site_series` , `get_latest_kpi_by_metric` , `get_latest_site_kpi_by_metric` , `get_latest_kpi` — passam a normalizar cada linha com `to_canonical(...)` antes de devolver.

Por que aqui e não em `api/api.py` : `get_kpi_series` da API tem múltiplos pontos de retorno (cluster, site fundido, família única) e converter em cada um deles é exatamente o tipo de duplicação que produz divergência. A camada de banco é o funil real, a conversão é linear, e toda agregação a jusante continua correta.

api/api.py

5. `get_kpi_catalog` e `get_kpi_overview` passam a devolver `base_unit` no lugar de `unit` . O `unit` bruto continua no payload, como `oss_unit` , para diagnóstico.

frontend/js/kpi.js

6. Remover os dois rótulos obsoletos de `METRIC_LABELS` (`"Volume de Tráfego DL (MB)" / "(UL)"`) e o fallback `" MB"` de `_getMetricSuffix` . Ambos mentem por 10^6 hoje: o valor é bit cru. Com `base_unit` vindo do backend, os fallbacks deixam de ter função.

Interface

Sem tela nova. O que muda é o **número exibido**, e a mudança é grande:

métrica	antes	depois
<code>traffic_volume_dl_nsa</code> (5G, site)	3.117.000	3.117.000.000
<code>throughput_dl</code> (4G, célula)	6,78	6.776.000

Os números ficam **piores de ler** ao fim desta fase. É esperado e transitório: a Fase 4 é quem formata. Vale rodar a Fase 3 e a Fase 4 juntas antes de mostrar para o usuário final.

Testes

arquivo	teste	o que trava
test_kpi_formulas.py	test_every_definition_declares_base_unit	nenhuma entrada do CATALOG com base_unit vazio
test_kpi_formulas.py	test_to_canonical_convert_5g_kbit_to_bit	1,0 kbit → 1000,0 bit
test_kpi_formulas.py	test_to_canonical_leaves_unknown_technology_untouched	technology=None → valor idêntico
test_kpi_formulas.py	test_4g_and_5g_volume_share_the_same_base_unit	as duas famílias devolvem "bit"
test_kpi_formulas.py	test_legacy_traffic_volume_keeps_monitoring_available	continua em catalog_for_api()
test_database.py	test_kpi_series_returns_canonical_values	linha 5G gravada em kbit sai em bit
test_database.py	test_rows_without_technology_are_returned_unchanged	banco sem a coluna não é convertido
test_api.py	test_overview_reports_base_unit_for_paired_panels	throughput_dl e throughput_ul com o mesmo base_unit

Como validar e testar

Testes automatizados

```
python -m pytest tests/ -q --basetemp=.pytest-work/fase3
```

Validação de não-regressão (a mais importante desta fase)

```
python tools/kpi_crosscheck.py --event testesantoamaro --csv-dir csvs_reference --raw
```

A flag `--raw` compara contra o **valor gravado**, não o convertido. Deve continuar `22/22` razão `1,000` — a Fase 3 não pode ter alterado nada no banco.

Em seguida, sem a flag, o script compara o valor **servido pela API** contra o CSV convertido para a unidade canônica. Também deve dar `22/22`.

Validação visual

1. Abrir a aba **5G** da visão geral. Confirmar que **Traffic Volume NSA** saltou de milhões para bilhões ($\times 1000$) e que **Throughput** saltou de unidades para milhões ($\times 10^6$).
2. Selecionar **Volume de Tráfego DL** no seletor de KPIs do dashboard. Confirmar que:
 - o rótulo não diz mais "(legado)" nem "(MB)";
 - a coluna **"Participação"** da lista de sites continua aparecendo e ordenando por volume — é o teste de regressão do B8.
3. Confirmar que percentuais, **dBm** e **UE médio** não mudaram de valor.

Sugestão de commit

```
refactor: normalize KPI units on read instead of migrating stored values
```

Fase 4 — Apresentação: escala automática e eixo único

O que será desenvolvido

Com todos os valores na mesma base, esta fase resolve a leitura. Três coisas:

1. **Escala automática por métrica (B4/B5):** a cada 1000, sobe de degrau — `bit` → `kbit` → `Mbit` → `Gbit`. Decimal (SI), não binário: `kbit` / `Mbit` / `Gbit` são potências de 10 por definição em telecom, ao contrário do `KiB` / `MiB` de sistema de arquivos.
2. **Eixo único nos painéis pareados (B3):** hoje DL vai para o eixo esquerdo e UL para o direito **mesmo quando a unidade é idêntica**, e os dois autoescalam sozinhos.
3. **“Sem tráfego” ≠ “Sem dados” (B6):** distinguir na tela o minuto em que a métrica é indefinida por ausência de tentativas do minuto em que não houve coleta.

Escopo detalhado

Código

`frontend/js/units.js` (arquivo novo)

```
1. export function escolherEscala(valores, unidadeBase) // -> { divisor, rotulo }
   export function formatar(valor, escala)           // -> string pt-BR
   export const ESCALAVEIS = new Set(["bit", "bit/s"]);
```

- Só `bit` e `bit/s` escalam (B5). `%`, `dBm`, `ms` e `usuários` passam direto.
- **Escala fixa por métrica no evento (B4):** o degrau é escolhido a partir do maior valor observado na métrica e **memorizado no `State`**, não recalculado por painel nem por refresh.
- **Histerese:** sobe de degrau em `>= 1000`, desce só em `< 900`.

Por que fixa, e não por painel. Medido nos dados do evento: no mesmo instante, o site `1774047` tem `33.460 kbit` e o `1774059` tem `2.489.072 kbit`. Com escala por painel, o primeiro exibiria `33,5 M` e o segundo `2,49 G` — e `33,5` **parece maior** que `2,49`. São 74× de diferença lidos ao contrário. Além disso, com janela deslizante de 15 min, o degrau do site `1774047` trocava sozinho **17 vezes em 427 pontos** (4,0% dos refreshes).

`frontend/js/kpi_overview.js`

2. **B3 — eixo único.** `_datasetFor` passa a decidir o eixo pela unidade, não pela direção:

```
const mesmaUnidade = new Set(panel.metrics.map(m => response.units?.[m])).size === 1;
yAxisID: (panel.paired && !mesmaUnidade && index === 1) ? "yRight" : "yLeft",
```

Os quatro painéis pareados do 5G têm unidade idêntica nas duas séries, então **todos passam a um eixo só**. O `yRight` fica no código para o caso de um par heterogêneo futuro.

3. **Corolário do B3 — o threshold volta a fazer sentido.** `_thresholdAnnotation` desenha em `yScaleID: "yLeft"` fixo. Com dois eixos, a linha tracejada do painel PRB Utility é comparada visualmente contra as barras de UL que estão em **outra escala** — o cruzamento que aparece na tela hoje é visualmente convincente e sem significado. Com eixo único, a comparação passa a ser válida sem tocar no código do threshold.

4. **unitLabel1 deixa de cair no fallback.** A linha `const unitLabel1 = units.length === 1 ? units[0] : "DL / UL"` produz "DL / UL" hoje porque `throughput_dl` declara `Mbit/s` e `throughput_ul` declara "unidade OSS pendente". Com a Fase 1 (unidade do UL resolvida) e a Fase 3 (`base_unit`), as duas passam a coincidir e o cabeçalho mostra a unidade de verdade. O fallback continua no código como rede.

5. `_formatNumber` passa a delegar para `units.formatar`, recebendo a escala do painel.

`frontend/js/kpi.js`, `frontend/js/vip.js`, `templates/report/report.js`

6. Os três formataores independentes passam a usar `units.js`. Sem isso, tela e relatório divergem — hoje já são quatro implementações diferentes (`toFixed(2)`, `toLocaleString`, e duas variantes).

`frontend/js/kpi_overview.js` + `frontend/index.html`

7. **B6 — rotulagem.** O overlay `.kpi-overview-no-data` passa a exibir:

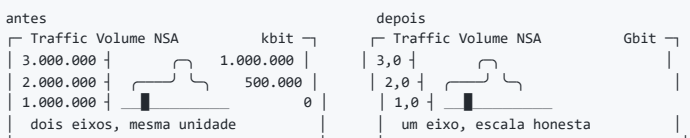
- "Sem dados no período" quando não houve coleta (nenhuma linha de nenhuma métrica);
- "Sem tráfego no período" quando houve coleta mas a métrica ficou indefinida.

O backend já tem a informação necessária desde a Fase 1 (`not_applicable`); basta o `get_kpi_overview` propagar um `reason` por métrica.

`frontend/js/bridge.js`

8. Atualizar os mocks: `unitsByMetric` deve refletir `base_unit` (`"bit"`, `"bit/s"`) e o mock de `get_kpi_overview` precisa gerar valores na ordem de grandeza canônica, senão os testes de Playwright validam uma escala que não existe em produção.

Interface



- Cabeçalho do painel Throughput deixa de exibir "DL / UL" e passa a exibir a unidade.
- Painel sem tráfego passa a dizer "Sem tráfego no período".

Testes

arquivo	teste	o que trava
test_frontend_units.py (novo)	test_escala_sobe_a_cada_mil	999 bit → bit ; 1000 → kbit ; 1e9 → Gbit
test_frontend_units.py	test_escala_e_decimal_nao_binaria	1024 bit → 1,02 kbit , nunca 1 KiB
test_frontend_units.py	test_histerese_evita_troca_de_degrau	1000 → sobe; cair para 950 não desce; 899 desce
test_frontend_units.py	test_percentual_e_dbm_nao_escalam	% e dBm passam intactos
test_frontend_collection_ui.py	test_paired_panel_uses_single_axis_when_units_match	Playwright: yRight ausente nos quatro pares 5G
test_frontend_collection_ui.py	test_panel_header_shows_unit_not_dl_ul	cabeçalho do Throughput ≠ "DL / UL"
test_frontend_collection_ui.py	test_empty_panel_distinguishes_no_traffic_from_no_data	os dois textos nos dois cenários
test_api.py	test_overview_reports_reason_per_metric	reason="no_traffic" quando not_applicable > 0

Como validar e testar

Testes automatizados

```
python -m pytest tests/ -q --basetemp=.pytest-work/fase4
```

Os testes de Playwright exigem playwright install chromium .

Validação visual (a fase com mais peso visual)

Abrir a visão geral com ?kpiTechnology=5G_NRDUCCELL no modo mock e conferir, painel a painel:

- 1. **PRB Utility** — eixo único. Hoje a barra de UL parece 20× a linha de DL; com eixo único a relação real (UL ~3× DL) fica visível. **A linha tracejada do threshold agora cruza as barras corretamente** — é a verificação mais importante da fase.
- 2. **Traffic Volume NSA** — eixo único, cabeçalho em Gbit , valores com 1–2 casas.
- 3. **Throughput** — cabeçalho com a unidade, não "DL / UL" .
- 4. **Estabilidade da escala** — deixar a tela aberta por 3 ciclos de coleta (~6 min) com a janela de 15 min. O rótulo da unidade **não pode trocar sozinho**.
- 5. **Comparação entre escopos** — trocar o seletor entre os dois sites do evento. Os dois painéis da mesma métrica devem exibir a **mesma unidade**, com o site menor aparecendo pequeno. É feio e é correto — a alternativa é bonita e engana.
- 6. **Relatório** — gerar um preview (tools/render_report_preview.py) e conferir que os valores saem na mesma unidade da tela.

Comparação lado a lado

Guardar capturas de antes/depois dos nove painéis 5G. A Fase 4 é a única cuja regressão é puramente visual e não é pega por teste.

Sugestão de commit

feat: scale KPI units automatically and share one axis per paired panel

Encaminhamentos fora deste plano

item	situação
A3	Não decidível com os dados coletados: os dois testes que rodei (eficiência espectral e teto de throughput) foram inconclusivos porque as células do SPSMH1 são pouco carregadas. A Fase 1 instala o aviso; a decisão exige olhar a configuração da célula no iManager ou comparar com o “DL PRB Usage” nativo do OSS. Não alterar a fórmula antes da resposta — se as células forem SFN com <code>Used</code> também somado, dividir por 75 criaria alarme falso onde hoje não há.
B6 (agregação)	A resposta foi “rotular agora e agregar depois”. A agregação por janela de exibição exige persistir os contadores brutos , que hoje não são gravados — é mudança de schema e merece plano próprio. É ela que devolve o alarme de acessibilidade ao 5G (ver Fase 1, item 13).
C1	Task 753 descarta ~1.650 objetos por ciclo; confirmado como configuração intencional. Nenhuma ação.
Banco legado	<code>smart_events_vips-rio-tim-jun-2026.db</code> (1.011.298 linhas) não tem as colunas <code>technology</code> e <code>scope</code> , e guarda volume em MB. Fica fora do contrato de unidade canônica (Fase 3, item 2). Se voltar a ser aberto, <code>init_event_db</code> adiciona as colunas com default vazio e as linhas antigas continuam sem conversão — comportamento correto, mas o histórico dele não é comparável com o dos eventos novos.

Atualizações de documentação exigidas

Ao fim de **cada** fase, conforme o `CLAUDE.md` :

- `MEMORY.md` — entrada datada com a decisão travada. Em especial: o `period` da resposta do OSS não é o GP (Fase 1); `N.ThpTime.*` medido como microssegundo e não documentado (Fase 1); a descontinuidade do throughput de site entre linhas antigas e novas (Fase 1); a opção por não migrar o histórico (Fase 3).
- `ERRORS.md` — entrada para os dois erros que passaram despercebidos e suas regras de prevenção: “*campo do OSS com nome plausível não é contrato — validar contra um contador que o denuncie*” (o `period=5` contra `N.Cell.Avail.Dur=60`) e “*contador sem unidade declarada não pode receber unidade por suposição — instalar trava aritmética*” (o `N.ThpTime.*`).